SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*
Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3

12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

*I **Mugesh S (192411055)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

ANSWERS

Q1. Testing as an Engineering Activity in a Banking Application

Answer:

Software testing should be treated as a systematic engineering activity rather than performing random tests. In a banking application, unstructured testing can lead to serious problems such as incorrect transactions, wrong account balances and unauthorized operations. To improve testing, the team should first understand the requirements and prepare a proper test plan. Test cases should be designed with clear inputs, test steps and expected results. Techniques such as equivalence partitioning, boundary value analysis, decision table testing and state transition testing can be used to cover different situations.

Testing tools can be introduced to manage test cases, automate repeated tests and track defects. Common testing standards and procedures should also be followed by all team members. Instead of ad-hoc testing, the team should use systematic test design, risk-based testing and regression testing.

For example, in a money transfer function, test cases should cover valid account numbers, invalid account numbers, sufficient balance, insufficient balance, transaction limits and network failures. Automation can repeatedly verify these functions whenever changes are made to the application.

A simple testing workflow is:

Requirement Analysis
↓
Test Planning and Test Case Design
↓
Test Execution
↓
Defect Reporting and Fixing
↓
Retesting and Regression Testing

This engineering approach improves test coverage, makes testing repeatable and helps detect defects early. It also reduces the possibility of serious banking failures and improves the overall quality, security and reliability of the application.

Q2. Structured Testing Process for ATM Withdrawal

Answer:

A structured testing process is necessary for an ATM system because it handles financial transactions and must produce accurate results. Testing should be performed throughout the Software Development Life Cycle (SDLC), beginning with requirement analysis and continuing until the software is ready for release.

During requirement analysis, the tester identifies requirements such as card validation, PIN verification, balance checking, withdrawal limits and cash dispensing. During test planning, the testing objectives, resources and schedule are defined. During test case design, cases are prepared for valid PINs, invalid PINs, insufficient balance, invalid withdrawal amounts and daily withdrawal limits. During test execution, actual results are compared with expected results. Any differences are recorded as defects, fixed by developers and checked again through retesting and regression testing.

For example, if a customer has ₹5,000 in their account and withdraws ₹2,000, the ATM should dispense ₹2,000 and update the remaining balance to ₹3,000. If the customer requests ₹6,000, the system should reject the transaction because the balance is insufficient.

A structured process helps ensure that all important conditions are tested consistently. It reduces missed defects, improves communication between testers and developers and provides better tracking of testing activities.

Thus, following a defined testing process improves the accuracy, reliability and security of ATM withdrawal functionality.

Q3. Error, Defect and Failure in a Web Application

Answer:

Error, defect and failure are three important concepts in software testing, and each has a different meaning. An error is a human mistake made during requirement analysis, design or coding. A defect is a flaw in the software introduced because of that mistake. A failure occurs when the software executes the defective code and produces an incorrect result.

For example, suppose a developer writes an incorrect condition while implementing login validation. The developer's mistake is an error. The incorrect condition in the login code is a defect. When a user enters an incorrect password and the application still allows access, the resulting incorrect behavior is a failure.

The relationship can be represented as:

Error

Human mistake during development

↓

Defect

Flaw in the software

↓

Failure

Incorrect behavior during execution

If the development team misunderstands these terms, defect reports may be unclear and the actual cause may take longer to identify. Therefore, testers should record the steps to reproduce the problem, expected output, actual output, severity and relevant environment details.

Understanding these concepts helps the team communicate clearly, identify root causes and improve software quality.

Q4. Defect Clustering and Test Case Prioritization

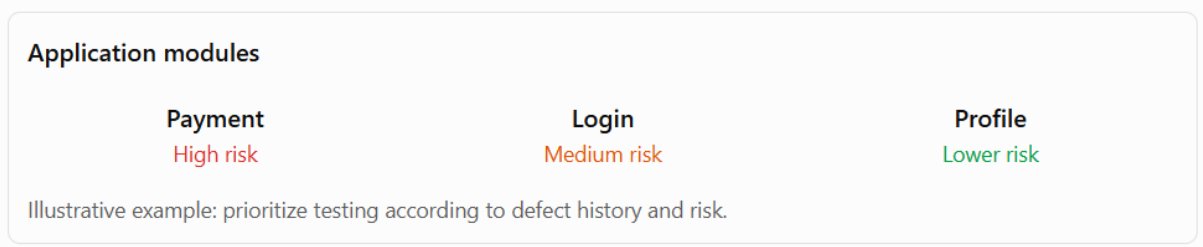
Answer:

Defect clustering is a software testing principle which states that a small number of modules often contain most of the defects. This happens because some modules are more complex, frequently modified or connected to many other modules. Therefore, testers should analyze previous defect reports and identify modules that have a high number of defects.

Test planning should be based on risk, complexity, defect history and business importance. Test cases for high-risk modules should be executed first. Critical functions should receive more attention than less important features. However, other modules should not be ignored completely.

For example, in an online banking application, the payment and transaction modules may contain more defects than the profile module. The testing team should prioritize transaction validation, balance calculation, duplicate transactions and payment failure scenarios.

The following diagram shows the concept:



Defect clustering helps use testing time and resources effectively. It improves defect detection, reduces the risk of serious failures and helps the team complete testing more efficiently.

Q5. Defect Repository and Defect Classification

Answer:

A defect repository is a centralized location used to record, classify, track and maintain information about software defects. Defects may originate from incorrect requirements, design mistakes, coding errors, integration problems or environmental issues. They can be classified according to their type, severity, priority, affected module and status.

For example, if a banking application transfers an incorrect amount, the defect can be recorded as a functional defect with high severity. The repository should contain the defect ID, description, steps to reproduce, expected result, actual result, severity, assigned developer and resolution details.

Testers should analyze historical defect data to identify repeated problems and common causes. If the payment module repeatedly contains defects, additional payment-related test cases should be included in future testing. Root cause analysis should also be performed to understand why these defects occur.

A defect repository supports better test case design, improves communication between developers and testers, and prevents recurring defects. Therefore, it is useful not only for tracking problems but also for improving the quality of future software releases.

Q6. Mobile Banking Application Crashes During Fund Transfer

Answer:

Frequent crashes during fund transfers indicate that the mobile banking application may contain functional, integration, performance or exception-handling defects. To identify the root cause, the tester should first reproduce the crash and examine application logs, server responses, network conditions and transaction details.

Systematic testing should cover successful transfers, insufficient balance, invalid account numbers, large transaction amounts, network disconnection and repeated transfer requests. The tester should also verify that the system handles interrupted transactions safely and does not debit the sender without correctly crediting the receiver.

For example, if the application crashes when the network disconnects during a transfer, the tester should check whether the transaction is completed, cancelled or left in an inconsistent state. The defect should be reported with reproduction steps, device details, application version and error logs.

After the developer fixes the issue, retesting and regression testing should be performed. Automated tests can also be introduced for important transaction scenarios.

This systematic approach helps identify root causes, prevents repeated failures and improves the reliability and security of mobile banking transactions.

Q7. Testing and Debugging

Answer:

Testing and debugging are related activities, but they serve different purposes. Testing is performed to identify defects or failures in software, whereas debugging is performed to locate the cause of a defect and correct it. Testers generally execute test cases and compare actual results with expected results, while developers investigate the code and fix the identified problems.

For example, suppose a banking application displays an incorrect account balance after a transaction. The tester reproduces the problem, records the expected and actual balances, and reports the defect. The developer then examines the transaction logic and database operations to identify the cause and correct the code.

If the team confuses testing with debugging, testers may fail to report defects properly and developers may struggle to reproduce the issue. Therefore, a structured approach should be followed: testers identify and report defects, developers debug and fix them, and testers perform retesting and regression testing.

Both activities are essential because testing detects problems and debugging helps remove their causes. Together, they improve software quality and reliability.

Q8. Cross-Browser Testing in an E-Commerce Platform

Answer:

Cross-browser testing verifies whether a web application works correctly across different browsers and versions. An e-commerce platform should be tested on supported browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox and Safari. Testing should cover login, product search, product details, shopping cart, payment and checkout functions.

Browser-related defects may occur because of differences in rendering engines, CSS support, JavaScript behavior and screen sizes. For example, the checkout button may work correctly in Chrome but may not respond properly in Firefox. Such a problem can affect customers and lead to incomplete purchases.

The testing team should prepare a browser compatibility matrix and execute important test cases across supported browsers and devices. Browser automation tools can reduce manual effort, while screenshots and browser logs can help identify defects.

Systematic cross-browser testing improves compatibility, prevents inconsistent behavior and provides a consistent user experience. It also helps ensure that customers can complete purchases successfully regardless of the supported browser they use.

Q9. Defect Classification in a Payroll System

Answer:

Incorrect salary calculations in a payroll system can be classified mainly as functional or logical defects because the system does not calculate salary according to the specified rules. Defects can also be categorized according to severity, priority, module and root cause.

For example, suppose an employee's basic salary is ₹30,000, but the system applies an incorrect deduction condition. The final salary will be incorrect even though the employee's details are correct. This is a logical defect that affects the payroll function and may have high severity if it impacts many employees.

Defect classification helps testers understand the nature and impact of a problem and prioritize important defects. Equivalence partitioning, boundary value analysis and decision table testing can be used to test salary ranges, deductions, overtime and tax calculations.

Code reviews, unit testing and regression testing can help prevent similar defects in future releases. Previous defects should also be stored in the defect repository and reused when designing test cases.

Proper defect classification improves testing effectiveness, reduces repeated calculation errors and increases confidence in the payroll system.

Q10. Structured Test Case Design

Answer:

Testing should be treated as a systematic engineering activity rather than executing random tests. The team should first analyze the requirements, identify test conditions and prepare test cases with clear inputs, steps and expected results. Each test case should have a unique ID so that it can be tracked and repeated whenever necessary.

Techniques such as equivalence partitioning, boundary value analysis, decision table testing and state transition testing can be used to improve test coverage. For example, if an application accepts an age between 18 and 60, the tester should check values such as 17, 18, 19, 59, 60 and 61.

After test case design, the cases should be executed and actual results should be compared with expected results. Defects should be recorded and retesting and regression testing should be performed after fixes.

Structured test case design makes testing repeatable, improves coverage and helps identify defects that random testing may miss. Therefore, it improves test effectiveness and overall software quality.

Q11. Data Inconsistency in a Healthcare Application

Answer:

Data inconsistency occurs when different modules of an application display or store different values for the same information. In a healthcare application, this may happen because of incorrect database operations, integration errors, synchronization problems or improper data validation.

For example, when a patient's phone number is updated in the profile module, the appointment module may continue displaying the old number. This can happen if the updated data is not correctly transferred between modules.

To identify the cause, testers should perform integration testing and database testing. They should verify data creation, modification, deletion and retrieval across all related modules. Expected database values should be compared with the values displayed by the application. Defects should be recorded with the affected modules, steps to reproduce, expected results and actual results.

A centralized defect repository helps track these issues until they are resolved. Retesting and regression testing should be performed after the fixes.

Systematic testing ensures data integrity, reduces inconsistencies and improves the reliability of the healthcare application.

Q12. Defect Repository and Repeated Defects Across Releases

Answer:

A defect repository helps identify and prevent repeated defects across software releases. If a company records defects but does not analyze them, the same problems may occur again because their root causes remain unresolved.

The repository should contain the defect ID, description, affected module, severity, priority, root cause, resolution and release details. Testers should analyze this information to identify patterns and determine which modules frequently contain defects.

For example, if several releases contain payment failures caused by incorrect database updates, the team should perform root cause analysis and introduce additional database validation and integration test cases. The previous failing test cases should also be added to the regression test suite.

Regular defect reviews, code reviews and preventive actions can reduce recurring problems. Historical defect information can also help prioritize high-risk modules during future test planning.

Thus, an effectively maintained defect repository improves test design, prevents repeated defects and increases software quality across releases.

Q13. Web-Based Learning System Fails During Peak Usage

Answer:

A web-based learning system that fails during peak usage indicates that important performance scenarios may not have been included in testing. The team should review its test plan and identify gaps in load testing, stress testing, concurrency testing and performance testing.

During test planning, the expected number of users, important functions and acceptable response times should be identified. For example, if thousands of students access an online examination system simultaneously, the application should be tested under a similar workload before deployment.

During performance testing, response time, throughput, server resource usage and system stability should be monitored. Load testing checks performance under expected workloads, while stress testing checks behavior beyond normal operating limits.

The team should also test important scenarios such as simultaneous login, exam submission and database access. Any performance defects should be recorded and corrected before release.

Proper test planning and coverage help identify performance bottlenecks early, reduce failures during peak usage and improve system reliability.

Q14. Early Involvement of Testers in SDLC

Answer:

Testers should be involved from the beginning of the Software Development Life Cycle instead of joining only after coding is completed. During requirement analysis, they review requirements and identify missing or ambiguous conditions. During design, they prepare test scenarios and participate in reviews. During development, they prepare test cases and test data. During integration and system testing, they verify that individual modules and the complete application work as expected.

For example, if a banking requirement does not clearly specify the maximum daily withdrawal amount, the tester can clarify it with the development team before coding begins. This prevents misunderstandings and reduces the cost of correcting defects later.

Early involvement also improves communication between developers, testers and business stakeholders. Regular meetings, requirement reviews and shared test documentation can strengthen collaboration.

Therefore, involving testers throughout the SDLC helps detect defects early, reduces rework and improves the overall quality of the software.

Q15. Defect Prevention Techniques

Answer:

Defect prevention focuses on identifying and removing the causes of defects instead of repeatedly fixing the same problems after they occur. When a software product experiences similar defects in every release, the team should perform root cause analysis to understand why they are being introduced.

Suitable prevention techniques include requirement reviews, design reviews, code reviews, coding standards, developer training, automated testing and regression testing. For example, if incorrect input validation repeatedly causes defects, the team can introduce common validation functions and include additional validation test cases.

Previous defect reports should be analyzed to identify common patterns. The team should also review whether the same mistakes occur because of unclear requirements, poor design or coding practices.

After implementing preventive measures, the team should verify their effectiveness in later releases. These practices reduce recurring defects, save time and effort, and improve software reliability and maintainability.

Q16. Structured Testing Strategy for a Mobile Application

Answer:

A mobile application that frequently crashes requires a systematic and standardized testing strategy. The team should first understand the requirements and prepare a test plan covering functional behavior, integration, performance, compatibility and regression testing.

Test cases should cover valid and invalid inputs, different devices, supported operating systems, screen sizes, network interruptions and unexpected user actions. For example, a mobile banking application should be tested during login, fund transfer, network disconnection and repeated transaction attempts.

The testing team should use standard test case formats, maintain proper test data and record defects with reproduction steps, device information and error logs. After fixing a crash, retesting and regression testing should be performed to ensure that the correction works without affecting other functions.

Automation can be introduced for frequently repeated and critical scenarios. This structured approach improves test coverage, ensures consistency and reduces the likelihood of crashes after deployment.

Q17. Testing and Debugging in Software Development

Answer:

Testing and debugging are two different activities that work together to improve software quality. Testing is performed to identify defects by comparing actual results with expected results, whereas debugging is performed to find the root cause of a defect and fix it.

For example, if a banking application displays the wrong account balance after a withdrawal, the tester executes the relevant test case, identifies the mismatch and reports the defect. The developer then examines the code, transaction logic and database updates to locate and correct the problem. After the fix, the tester performs retesting and regression testing.

Confusing testing with debugging may delay defect resolution because the problem may not be reported clearly or investigated by the appropriate person. A structured process should therefore assign defect identification and reporting to testers and debugging and fixing to developers, followed by verification.

Testing helps discover problems, while debugging removes their causes. Both activities are essential for producing reliable and high-quality software.

Q18. Cross-Browser Testing in an E-Commerce Platform

Answer:

Cross-browser testing ensures that an e-commerce platform behaves consistently across different browsers and versions. The testing team should identify supported browsers such as Chrome, Edge, Firefox and Safari and prepare test cases for important functions such as login, product search, cart management, payment and checkout.

Defects may occur because browsers interpret HTML, CSS and JavaScript differently. For example, a checkout button may appear correctly in Chrome but become misaligned or stop responding in Firefox. This can prevent customers from completing purchases.

The team should execute functional and visual test cases across supported browsers, operating systems and screen sizes. A browser compatibility matrix can help track test coverage, and automation tools can reduce repetitive manual testing. Defects should be recorded with browser details and reproduction steps, then fixed and retested.

A systematic cross-browser testing strategy improves compatibility, reduces customer dissatisfaction and provides a consistent shopping experience.

Q19. Defect Reporting in a Logistics Application

Answer:

Poor defect reporting can delay defect identification and resolution in a logistics application. To improve the process, testers should clearly identify the problem and record the expected and actual results. A defect report should include a unique ID, description, affected module, steps to reproduce, input data, severity, priority and relevant environment details.

For example, if the system calculates an incorrect delivery charge for a particular distance, the tester should record the distance entered, expected delivery charge and actual charge displayed. This information helps the developer reproduce the issue and identify the cause.

Defect reports should be maintained in a centralized tracking system so that their status can be monitored from identification to closure. Testers should also verify the fix through retesting and perform regression testing on related functions.

Clear defect reporting improves communication, reduces misunderstandings and helps resolve problems faster, improving the reliability of the logistics application.

Q20. Redesigning Testing for a Banking System

Answer:

Repeated failures in banking transaction processing indicate that the existing testing practices are insufficient or inconsistent. The testing process should be redesigned using systematic test planning, structured test case design, proper defect tracking and regression testing.

First, the requirements should be analyzed to identify important transaction conditions. Test cases should cover successful transfers, insufficient balance, invalid account details, duplicate requests, transaction limits, network failures and cancelled transactions. Techniques such as equivalence partitioning, boundary value analysis and decision table testing can improve coverage.

For example, when ₹5,000 is transferred from one account to another, the sender's balance should decrease by ₹5,000 and the receiver's balance should increase by ₹5,000. If the transaction fails, the system should avoid inconsistent partial updates.

Defects should be recorded in a repository, and repeated failures should be analyzed to identify their root causes. Automated regression testing should be used for critical transaction scenarios.

This structured approach improves defect detection, reduces repeated transaction failures and increases the accuracy, reliability and security of the banking system.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand